

Comandos *principais*

Os comandos que você digita no terminal para usar o Codex no dia a dia: iniciar e retomar sessões, autenticar, automatizar com `exec`, revisar código, conectar servidores MCP, trazer trabalho da nuvem e diagnosticar problemas.

`codex`

`resume · fork`

`exec · review`

`mcp · cloud · apply`

`doctor · update`

Dois tipos de *comando*

O Codex tem **subcomandos de terminal** (`codex exec` , `codex resume` ...), digitados no shell, e **slash commands** (`/model` , `/review` ...), digitados dentro da sessão. Esta aula cobre os subcomandos. Os slash commands são o tema da próxima aula.

SESSÃO INTERATIVA

codex · resume · fork

Abrir a interface, retomar um chat salvo ou bifurcar uma conversa.

AUTOMAÇÃO

exec · review

Rodar tarefas e code reviews sem interação, em scripts e no CI.

CONTA

login · logout · update

Autenticação, logout e atualização da CLI.

INTEGRAÇÕES

mcp · plugin · cloud · apply

Servidores MCP, plugins e o fluxo com o Codex Cloud.

MANUTENÇÃO

doctor · features · completion

Diagnóstico, feature flags e autocompletar no shell.

ORGANIZAÇÃO

archive · unarchive · delete

Arquivar, restaurar e apagar sessões salvas.

Na dúvida, pergunte à própria CLI: `codex --help` lista tudo, e `codex <subcomando> --help` mostra as opções de cada um. Os marcados como `experimental` na documentação podem mudar entre versões.

codex: abrindo a interface interativa

Sem subcomando, `codex` abre a interface de terminal (TUI) no diretório atual. Ele aceita um prompt inicial, imagens e as flags globais.

```
# abre a TUI no projeto atual
codex

# já começa com um pedido
codex "Explique a estrutura deste projeto"

# anexa um print do erro ao primeiro prompt
codex -i erro.png "O que causa este erro?"

# outro diretório + modelo específico
codex -C ~/projetos/api -m gpt-6-sol

# edita o workspace, pede aprovação para o resto
codex -s workspace-write -a on-request

# busca na web ao vivo para dúvidas atuais
codex --search "Qual a versão estável mais recente do Vite?"
```

FLAG	PARA QUÊ
<code>-m</code> <code>--model</code>	Escolhe o modelo da sessão
<code>-i</code> <code>--image</code>	Anexa imagens ao primeiro prompt
<code>-C</code> <code>--cd</code>	Define o diretório de trabalho
<code>-s</code> <code>--sandbox</code>	<code>read-only</code> , <code>workspace-write</code> ou <code>danger-full-access</code>
<code>-a</code> <code>--ask-for-approval</code>	<code>on-request</code> ou <code>never</code>
<code>--search</code>	Busca na web ao vivo
<code>--add-dir</code>	Libera escrita em outra pasta
<code>-c chave=valor</code>	Sobrescreve o config.toml só nesta execução

Para liberar mais pastas, prefira `--add-dir` a `danger-full-access` . É a recomendação da própria documentação.

Login, logout e *atualização*

Na primeira execução, o `codex` já pede login. Os subcomandos abaixo servem para os outros cenários: servidor sem navegador, chave de API, máquina compartilhada.

```
# login pela conta do ChatGPT (abre o navegador)
codex login

# sem navegador: código de dispositivo (SSH, VPS)
codex login --device-auth

# chave de API lida do stdin (nunca como argumento)
printenv OPENAI_API_KEY | codex login --with-api-key

# qual modo está ativo? (sai com 0 se logado)
codex login status

# remove as credenciais salvas
codex logout
```

CODEx UPDATE

Verifica e aplica atualizações da CLI quando a instalação permite auto-update. Se você instalou por npm ou Homebrew, atualize por eles: `npm install -g @openai/codex` ou `brew upgrade --cask codex`.

CODEx APP

Abre o app desktop do ChatGPT (aula 2 do módulo anterior). No macOS, `codex app ~/projeto` já abre a pasta. No Windows, o comando mostra o caminho para abrir.

Lendo a chave pelo stdin, ela não fica no histórico do shell nem aparece na lista de processos.

Retomar, bifurcar e *organizar sessões*

Toda sessão interativa fica salva. Assim você volta de onde parou, experimenta outro caminho sem perder o original e mantém a lista organizada.

```
# seletor de sessões do diretório atual
codex resume

# retoma direto a mais recente
codex resume --last

# inclui sessões de outros diretórios
codex resume --all

# copia uma sessão para um chat novo (o original fica intacto)
codex fork --last

# esconde do seletor, sem apagar · restaura
codex archive "refatoracao-login"
codex unarchive "refatoracao-login"

# apaga de vez (--force só aceita UUID)
codex delete <SESSION_UUID> --force
```

COMANDO	QUANDO USAR
<code>resume</code>	Continuar uma tarefa no dia seguinte, com todo o contexto
<code>fork</code>	Testar uma abordagem alternativa a partir do mesmo ponto
<code>archive</code>	Limpar o seletor sem perder a transcrição
<code>delete</code>	Remover a transcrição definitivamente

Sessões são identificadas por **ID** ou **nome**, e o ID tem prioridade. Para nomear, use `/rename` dentro da sessão (próxima aula).

O `delete` é irreversível. Na dúvida, prefira `archive`.

codex exec: tarefas sem interação

`codex exec` (alias `codex e`) roda uma tarefa do início ao fim sem pedir nada ao usuário. Serve para scripts, hooks do Git e pipelines de CI.

```
# tarefa com escrita no workspace
codex exec -s workspace-write "Adicione type hints em
src/utils.py"

# prompt vindo de um arquivo (stdin)
cat pedido.md | codex exec -

# CI: eventos JSONL + resumo final num arquivo
codex exec --json -o resumo.md "Rode os testes e explique as
falhas"

# resposta final validada por um JSON Schema
codex exec --output-schema schema.json "Liste as dependências
desatualizadas"

# continua a última execução deste diretório
codex exec resume --last "Agora corrija a primeira falha"
```

OPÇÃO	EFEITO
<code>--json</code>	Um evento JSON por linha
<code>-o</code> <code>--output-last-message</code>	Grava só a resposta final
<code>--ephemeral</code>	Não salva a sessão em disco
<code>--skip-git-repo-check</code>	Roda fora de um repositório Git
<code>--ignore-user-config</code>	Ignora o config.toml pessoal

Dica da documentação: no CI, combine `--json` (progresso legível por máquina) com `-o` (resumo em linguagem natural).

`--full-auto` está descontinuado. Use `-s workspace-write .`

codex review: code review no terminal

Roda uma revisão de código **sem interação** e sem alterar a sua working tree. Escolha **um único alvo** por execução: as opções abaixo não se combinam entre si.

```
# tudo que ainda não foi commitado
# (staged, unstaged e arquivos novos)
codex review --uncommitted

# o branch atual comparado com a main
codex review --base main

# as mudanças de um commit específico
codex review --commit a1b2c3d --title "Login com OAuth"

# instruções próprias de revisão
codex review "Procure problemas de segurança nas queries SQL"
```

1 Antes do commit

`codex review --uncommitted` pega problemas enquanto ainda é barato corrigir.

2 Antes do PR

`codex review --base main` revisa o branch inteiro, como um revisor humano faria.

3 Depois do merge

`codex review --commit <sha>` audita uma mudança específica.

`--title` só funciona junto com `--commit`. Para revisar dentro de uma sessão interativa, use o slash command `/review`.

codex mcp: conectando ferramentas externas

Gerencia os servidores **Model Context Protocol** guardados em `~/.codex/config.toml`. São os mesmos servidores MCP que você já usa no Claude Code.

```
# servidor stdio (ex.: Context7, docs de bibliotecas)
codex mcp add context7 -- npx -y @upstash/context7-mcp

# stdio com variáveis de ambiente
codex mcp add meu-db --env DB_URL=postgres://... -- node
server.js

# servidor HTTP remoto (streamable HTTP)
codex mcp add docs --url https://mcp.exemplo.com/mcp

# listar · ver · remover
codex mcp list --json
codex mcp get context7
codex mcp remove meu-db

# OAuth (só servidores HTTP que suportam)
codex mcp login docs
```

TRANSPORTE	COMO REGISTRAR
stdio	Comando e argumentos depois de <code>--</code> , com <code>- -env</code> opcional
HTTP	<code>--url</code> , com <code>--bearer-token-env-var</code> ou OAuth

CODEx PLUGIN

Instala, lista e remove **plugins** dos marketplaces configurados. Os marketplaces são gerenciados com `codex plugin marketplace` (fontes Git ou locais).

O token de um servidor HTTP deve ficar numa **variável de ambiente**, e você passa só o nome dela em `--bearer-token-env-var`, nunca o valor.

codex cloud e *codex apply*

Estes comandos ligam o terminal ao Codex Cloud: você despacha tarefas para um environment na nuvem e depois traz o diff para o repositório local.

```
# seletor interativo de chats e environments
codex cloud

# envia uma tarefa (2 tentativas para comparar)
codex cloud exec --env ENV_ID --attempts 2 "Migre os testes para
pytest"

# chats recentes, em JSON para scripts
codex cloud list --json

# aplica o diff do chat na sua working tree
codex apply TASK_ID
```

1

Despache

`codex cloud exec` manda a tarefa e libera o seu terminal.

2

Acompanhe

`codex cloud list` mostra o status, e a revisão pode ser feita no navegador.

3

Traga para casa

`codex apply` (alias `codex a`) aplica o diff localmente com `git apply`.

`codex cloud` é **experimental**. O `codex apply` encerra com erro se o `git apply` falhar por conflito. Faça commit ou stash do seu trabalho antes.

Diagnóstico e *ajustes finos*

Comandos para quando algo não funciona, ou quando você quer deixar a CLI do seu jeito.

```
# relatório de saúde: instalação, config, login, Git...
codex doctor
codex doctor --summary          # só o resumo
codex doctor --json             # versão sem dados sensíveis, para
                                # suporte

# feature flags (gravadas no config.toml)
codex features list
codex features enable <nome>
codex features disable <nome>

# autocompletar no zsh (também bash, fish, powershell)
codex completion zsh > "${fpath[1]}/_codex"

# roda um comando sob o sandbox do Codex
codex sandbox linux -- ls ~/.ssh
```

CODEX DOCTOR

O **primeiro comando** a rodar quando algo der errado: ele verifica instalação, configuração, autenticação, runtime, Git, terminal e sessões.

CODEX SANDBOX

Executa qualquer comando sob as mesmas regras que o agente usa (Seatbelt no macOS, Landlock no Linux, sandbox nativo no Windows). É ótimo para entender por que o agente foi bloqueado.

Também existe o `codex execpolicy` (experimental), que testa arquivos de regras e mostra se um comando seria permitido, perguntado ou bloqueado.

Cola: *todos os comandos* numa página

COMANDO	O QUE FAZ
<code>codex</code>	Abre a interface interativa
<code>codex resume</code>	Retoma uma sessão salva
<code>codex fork</code>	Bifurca uma sessão num chat novo
<code>codex exec</code> · <code>e</code>	Tarefa não interativa (scripts e CI)
<code>codex review</code>	Code review não interativo
<code>codex login</code> · <code>logout</code>	Autenticação
<code>codex update</code>	Atualiza a CLI
<code>codex app</code>	Abre o app desktop do ChatGPT
<code>codex archive</code> · <code>unarchive</code>	Esconde ou restaura uma sessão
<code>codex delete</code>	Apaga uma sessão (irreversível)

COMANDO	O QUE FAZ
<code>codex mcp</code>	Servidores MCP (add, list, get, remove, login)
<code>codex plugin</code>	Plugins e marketplaces
<code>codex cloud</code> <code>exp.</code>	Chats do Codex Cloud pelo terminal
<code>codex apply</code> · <code>a</code>	Aplica o diff de um chat da nuvem
<code>codex doctor</code>	Relatório de diagnóstico
<code>codex features</code>	Lista, liga e desliga feature flags
<code>codex completion</code>	Autocompletar para o shell
<code>codex sandbox</code>	Roda um comando sob o sandbox do Codex
<code>codex execpolicy</code> <code>exp.</code>	Testa regras de execução
<code>codex app-server</code> <code>exp.</code>	Servidor para integrar o Codex em apps

Teste *rápido*

1. Você fechou o terminal ontem no meio de uma tarefa. Como voltar direto para a última sessão deste projeto?

A) `codex fork --all`

B) `codex resume --last`

C) `codex exec resume`

D) `codex unarchive`

B. O `codex resume --last` pula o seletor e reabre a sessão mais recente do diretório atual.

2. Qual comando revisa todo o branch atual antes de abrir um PR para a main?

A) `codex review --uncommitted`

B) `codex review --commit main`

C) `codex review --base main`

D) `codex exec --review`

C. O `--base main` compara o branch atual com a main. O `--uncommitted` olha só o que ainda não foi commitado.

3. Como registrar um servidor MCP stdio no Codex?

A) `codex mcp add nome --url npx ...`

B) `codex mcp add nome -- npx -y pacote`

C) `codex plugin add nome`

D) `codex features enable mcp`

B. Nos servidores stdio, o comando que inicia o servidor vem depois de `--`. O `--url` é para servidores HTTP.

4. O Codex está se comportando de forma estranha depois de uma atualização. Qual o primeiro comando a rodar?

A) `codex doctor`

B) `codex logout`

C) `codex delete --force`

D) `codex sandbox`

A. O `codex doctor` verifica instalação, configuração, autenticação, Git e terminal, e aponta o que está errado.

○ RECAPITULANDO

Do terminal ao CI, *um comando por tarefa*

Com o `codex` você abre a sessão, com `resume` e `fork` retoma e experimenta, com `exec` e `review` automatiza, com `mcp` conecta ferramentas, com `cloud` e `apply` delega para a nuvem e com `doctor` resolve problemas. Quando precisar lembrar de uma opção, use `codex --help`.

`codex`

`resume · fork`

`exec · review`

`mcp`

`cloud · apply`

`doctor`

Próxima aula → Slash Commands

Fontes: [learn.chatgpt.com/docs](https://learn.chatgpt.com/docs/codex/cli) (`codex/cli` · `developer-commands?surface=cli` · `extend/mcp` · `cloud` · `non-interactive-mode`). Comandos marcados como experimentais podem mudar; confira `codex --help` na sua versão.